Vaidya AI VEDAI REPOSITORY

RAG SYSTEM · TECHNICAL HANDOVER

06

# Installation & Deployment Guide

Local setup, release packaging and operational runbooks

DOCUMENT / APPLICATION
**Version 1.0 / VedAI 0.7.0**

PREPARED
**11 November 2026**

AUDIENCE
**Engineering & release operations**

STATUS
**Handover draft · source verified**

Repository: github.com/AmRitJain0442/vedai
Source revision: 73beef60490f97bfbd2902db152ff99a6484456d

# Contents

VAI-HO-06 · Installation & Deployment Guide

1. Document control and operating assumptions 3
2. Prerequisites and dependencies 3
3. Fresh local installation 4
4. Environment configuration 6
5. Optional semantic/hybrid retrieval 7
6. Build and release artifacts 7
7. Recorded deployment topology 8
8. Initial Linux deployment runbook 9
9. Service behavior and routine operations 10
10. Code update and rollback 10
11. Validation and release acceptance 11
12. Troubleshooting 12
13. Backup, shutdown and handover 13
14. Source references 13

Prepared from the pinned implementation. Local corpus, default retrieval and UI were verified. Live hosting and paid model generation were not revalidated. See the verification record for scope.

# 1. Document control and operating assumptions

| Control | Value |
|---|---|
| Document ID / version | VAI-HO-06 / 1.0 — implementation-based handover draft |
| Prepared | 11 November 2026 |
| Target application | Vaidya AI / `vedai` 0.7.0 |
| Inspected revision | `73beef60490f97bfbd2902db152ff99a6484456d` |
| Audience | Developers, release engineers, operators and receiving maintainers |
| Locally verified | Windows/Python install, complete ingestion, default indexes, API retrieval and browser workflows |
| Not executed for this handover | GCP provisioning, Linux installation, live provider calls, Ollama embedding build |

Commands below assume a repository checkout and access to the upstream corpus. Linux hosting steps are derived from the committed scripts and service files; they are a runbook for the receiving operator, not a claim that the live infrastructure was inspected or redeployed. Replace example hostnames, usernames and paths with the approved environment values before execution.

# 2. Prerequisites and dependencies

| Requirement | Local default reading room | Optional / hosted use |
|---|---|---|
| Python | 3.10 or newer | Linux bootstrap installs distribution Python 3 |
| SQLite | FTS5, trigram tokenizer and JSON functions | Needed for all default runtime indexes |
| Git | Clone source and pin a commit | Packaging enumerates Git-visible files |
| Network | GitHub corpus archive and Python package index | OpenRouter for generation; model download for Ollama |
| Python extras | `.[rag,test]` recommended for setup/verification | `.[rag]` for runtime installation |
| Browser | Modern browser with JavaScript and IndexedDB | Clipboard features need an allowed secure context |
| OpenRouter key | Not needed for retrieval | Enables planning, generated claims and source translations |
| Ollama | Not needed for expanded mode | Required for embed/semantic/hybrid modes |
| Linux/systemd | Not needed locally | Required by provided VM deployment scripts |
| GCP CLI/IAP access | Not needed locally | Required for the recorded GCP deployment workflow |

No Node.js, npm build, Docker image, PostgreSQL service or cloud vector store is required. Core download/process/lexical commands have no third-party package dependency. The web application needs the `rag` extra, which installs NumPy, FastAPI, Uvicorn and indic-transliteration. The `test` extra adds httpx. Playwright is an optional separate installation for browser smoke tests.

For capacity planning, the recorded deployment uses an `e2-small` VM and a 20 GB standard persistent disk. This is a historical configuration, not a measured minimum or service-level sizing commitment. Allow additional disk capacity for extracted XML, JSONL artifacts, several SQLite indexes, staging packages and backups. The full corpus archive is about 32 MiB and extracts to about 171 MiB; processed files/indexes add substantially more.

# 3. Fresh local installation

## 3.1 Clone and select the handover revision

```
git clone https://github.com/AmRitJain0442/vedai.git
Set-Location vedai
git checkout 73beef60490f97bfbd2902db152ff99a6484456d
python --version
```

The checkout command selects the documented snapshot. To develop on it, create an appropriate working branch. Run all subsequent commands from the repository root so that default `data`, `config`, `reports` and `.env` paths resolve correctly.

## 3.2 Create an isolated Python environment

Windows PowerShell, without requiring activation-policy changes:

```
python -m venv .venv
.\.venv\Scripts\python.exe -m pip install ".[rag,test]"
```

Linux/macOS:

```
python3 -m venv .venv
.venv/bin/python -m pip install '.[rag,test]'
```

In the remaining local examples, `python` means the interpreter inside this environment. Activate it if convenient, or substitute its full path. The handover's separate reference file records the resolved package versions used for verification. The repository supplies version ranges rather than a lockfile.

## 3.3 Check SQLite capabilities

```
python -c "import sqlite3; d=sqlite3.connect(':memory:'); d.execute('CREATE VIRTUAL TABLE probe USING
fts5(text, tokenize=trigram)'); print(sqlite3.sqlite_version); print(d.execute('SELECT
json_valid(?)', ('{}',)).fetchone())"
```

Expected: a SQLite version and `(1,)`. An error about FTS5, tokenizer or JSON means this Python SQLite build is unsuitable. Install a Python distribution/build with the required SQLite features before indexing.

## 3.4 Download and process the corpus

```
python -m vedai download
python -m vedai process
python -m vedai index
python -m vedai terms-index
python -m vedai passages-index
python -m vedai verify
```

Keep this order: process requires the source manifest/extracted XML; lexical and terms indexes require processed chunks; passage indexing requires processed segments. The source selection is `config/ayurveda.json`. Data is ignored by Git and is not shipped in a normal clone.

Expected pinned-build results:

| Check | Expected result |
|---|---|
| Download | 110 source files, including 88 XML files |
| Process | 15 works, 145,600 segments, 26,585 chunks |
| Lexical/terms index | 26,585 chunks each |
| Passage index | 111,195 complete source units |
| Verify | 110 file hashes, 145,600 unique segments, 26,585 unique chunks, 119,774 citation spans; coverage and lexical IDs verified |

Processing/indexing regenerates derived files. For a nondefault data location put the global option **before** the command on every invocation:

```
python -m vedai --data D:\VaidyaData download
python -m vedai --data D:\VaidyaData process
python -m vedai --data D:\VaidyaData index
python -m vedai --data D:\VaidyaData terms-index
python -m vedai --data D:\VaidyaData passages-index
python -m vedai --data D:\VaidyaData verify
python -m vedai --data D:\VaidyaData serve --port 8765
```

Reports still default to repository-relative paths unless a command supports a separate report option. Avoid mixing artifacts from different data roots.

## 3.5 Start and validate

```
python -m vedai serve --port 8765
```

Open `http://127.0.0.1:8765`. The server binds to loopback and defaults to expanded retrieval. If `--port` is omitted, the CLI default is **8000**, despite README/service examples using 8765.

In another terminal:

```
Invoke-RestMethod http://127.0.0.1:8765/health
python -m vedai ask "What are the three doshas?" --work astangahrdayasamhita --retrieval-only
python -m unittest discover -s tests -v
```

Check that health body `status` is `ready`, catalog contains 15 works, and retrieval returns source passages without a key. `generation_configured=false` is expected in retrieval-only setup. A health HTTP 200 alone is not sufficient; inspect its body and run a real ask.

# 4. Environment configuration

## 4.1 Supported variables

| Variable | Default / accepted value | Source / effect |
|---|---|---|
| `OPENROUTER_API_KEY` | Empty | Key for hosted model requests |
| `OPENROUTER_MODEL` | `openai/gpt-5.6-sol` | Repository default model string; availability not live-tested here |
| `OPENROUTER_REASONING_EFFORT` | `medium` for default Sol; model-dependent fallback | `none`, `low`, `medium`, `high`, `xhigh`, `max`, or absent |
| `OLLAMA_HOST` | `http://127.0.0.1:11434` | Optional local embedding server URL |
| `VEDAI_ALLOWED_HOSTS` | No extra hosts | Comma-separated external Host names appended to built-ins |
| `VEDAI_ACCESS_USER` | `vedai` | Shared application Basic-auth username |
| `VEDAI_ACCESS_PASSWORD` | Empty | Nonempty enables Basic auth on all routes |
| `VEDAI_REQUIRE_AUTH` | Disabled | Exactly `1` requires a configured access password at startup |
| `PYTHONDONTWRITEBYTECODE` | `1` in systemd unit | Avoids runtime `.pyc` writes |

Only the **three OpenRouter variables** are automatically read from the repository-root `.env` by `local_settings()`. Ollama and `VEDAI_*` settings must be actual process environment variables, or be provided through systemd's `EnvironmentFile`. Putting them only in the local `.env` does not configure them. Process environment overrides `.env`; the parser does not execute shell code or expand variable references. Restart after configuration changes.

## 4.2 Enable generated answers locally

```
Copy-Item .env.example .env
```

Edit the file privately:

```
OPENROUTER_API_KEY=REPLACE_WITH_PRIVATE_KEY
OPENROUTER_MODEL=openai/gpt-5.6-sol
```

```
OPENROUTER_REASONING_EFFORT=medium
```

The selected model must support the application's strict JSON-schema response contract. Generation normally makes a planning call and an answer/translation call. To reduce scope for a particular request, use `enhance=false` or `generate=false`; these are request controls, not environment variables. The adapter does not automatically retry paid calls. `generation_configured=true` only confirms nonempty settings, not working credits, valid credentials or model availability.

### 4.3 Configure local access/hosts when needed

```
$env:VEDAI_ALLOWED_HOSTS='demo.example.org'
$env:VEDAI_ACCESS_USER='vedai'
$env:VEDAI_REQUIRE_AUTH='1'
```

Supply `VEDAI_ACCESS_PASSWORD` through a private process-launch mechanism or protected environment configuration, then start the server. Do not commit credentials or copy them into a general release manifest. Local health/API tools must provide Basic credentials when authentication is enabled.

## 5. Optional semantic/hybrid retrieval

This path is unnecessary for the default reading room and remains experimental in repository evaluation. Install/start Ollama separately, then:

```
ollama pull embeddinggemma
python -m vedai embed --model embeddinggemma --batch-size 32
python -m vedai serve --port 8765 --mode hybrid
```

`semantic` is another supported mode. `hybrid` requires terms plus complete compatible vectors. Only the EmbeddingGemma model family has an implemented prompt profile. The builder permits batch sizes 1–128, caches completed batches in SQLite, disables truncation, and stops explicitly for overlong chunks. Retrying an unchanged build resumes cached rows.

If corpus hash, model digest or prompt version changes, use a new coherent data directory or move aside **both** vector database and manifest before rebuilding. Do not mark a partial manifest complete manually. Runtime semantic retrieval checks source/hash/model compatibility and normalized matrix dimensions. It loads vectors into memory, so size capacity accordingly. The historical embedding report is not proof that a freshly deployed host already has Ollama/model data.

## 6. Build and release artifacts

The application has no frontend compilation step: static files are packaged as Python package data. A deployable wheel can be built with:

```
python -m pip wheel --no-deps . --wheel-dir dist
```

Install the source with extras for a runtime host, or install a wheel with its appropriate extras. The supplied VM workflow uses a source release archive rather than a wheel-only release.

### 6.1 Full release

After the required data/index build:

```
python deploy/package.py
Get-Content data/runtime/release-manifest.json
Get-FileHash data/runtime/vedai-release.tar.gz -Algorithm SHA256
```

The script creates `data/runtime/vedai-release.tar.gz` and `release-manifest.json`. It collects Git-visible repository files and selected corpus artifacts: files currently in `data/index`, processed headers, `works.json`, chunks/segments, source manifest, and exactly one pinned raw ZIP. It includes optional vector artifacts only if present. A fresh default build does not automatically create vectors.

The full archive does not include the entire extracted raw directory or per-work processed JSONL directory. Runtime reads use the packaged combined files. To run `verify` on a fresh release-only host, restore/extract the pinned raw archive first, for example by running the repository downloader against that data root. The installer does not perform ingestion or create missing indexes.

`.env` is rejected, and ignored `data/runtime` files are not selected. Nevertheless, inspect `release-manifest.json`: Git-visible, nonignored files are included broadly, so do not rely on the package builder to identify arbitrary secret filenames elsewhere in the checkout. Use a clean release checkout and keep deployment credentials external.

### 6.2 Code-only release

```
python deploy/package.py --code-only
Get-Content data/runtime/code-manifest.json
```

This produces `vedai-code.tar.gz`, omitting corpus/index data. Use it only when existing data and dependency installations remain compatible. A model/pipeline/schema/dependency change may require a full staged release or additional controlled data rebuild. Archive checksums identify the transferred artifact; they do not substitute for review of its contents.

## 7. Recorded deployment topology

The repository documents Debian 12 on a dedicated Compute Engine VM:

| Item | Recorded value / purpose |
|---|---|
| Project / VM / zone | `tribe-v2-host` / `vedai-web` / `us-central1-a` |
| Machine / disk | `e2-small` / 20 GB standard persistent disk |
| Network / subnet | `vedai-net` / `vedai-us-central1`, `10.67.0.0/24` |
| Inbound access | SSH through Google IAP; app origin remains loopback |
| GCP service account | None attached in recorded setup |
| App unit | `vedai.service`, unprivileged OS user `vedai` |
| Origin | `http://127.0.0.1:8765` |

| Item | Recorded value / purpose |
|---|---|
| Connector | `vedai-tunnel.service`, outbound Cloudflare Quick Tunnel |
| Private environment | `/etc/vedai.env`, root-owned, mode 0600 |

These are identifiers from committed deployment notes, not a current infrastructure inventory. The scripts do not provision the GCP project, VM, subnet, firewall or IAP IAM permissions. An infrastructure operator must supply that approved host and SSH access before bootstrap. No Docker/Kubernetes/managed-platform deployment is provided.

# 8. Initial Linux deployment runbook

## 8.1 Prepare, transfer and verify

Build and inspect the full release locally. Privately prepare a Linux/systemd-compatible environment file containing the OpenRouter values, allowed external hostname and, if selected, application access values. Store it outside the repository and stage it with restricted local permissions. The install script expects the destination name `/tmp/vedai.env`.

Example transfer to the recorded VM:

```
gcloud compute scp data/runtime/vedai-release.tar.gz deploy/bootstrap.sh deploy/install.sh vedai-web:/tmp/ --project=tribe-v2-host --zone=us-central1-a --tunnel-through-iap
gcloud compute scp PRIVATE_ENV_FILE vedai-web:/tmp/vedai.env --project=tribe-v2-host --zone=us-central1-a --tunnel-through-iap
gcloud compute ssh vedai-web --project=tribe-v2-host --zone=us-central1-a --tunnel-through-iap
```

Replace `PRIVATE_ENV_FILE` with the actual protected path. Before installation, compare the transferred release hash to the recorded local manifest:

```
sha256sum /tmp/vedai-release.tar.gz
chmod 600 /tmp/vedai.env
sudo bash /tmp/bootstrap.sh
sudo bash /tmp/install.sh
```

`bootstrap.sh` installs Python/venv, certificates and curl; creates the unprivileged `vedai` user/directory; downloads cloudflared **2026.9.1** and checks the exact committed SHA-256 before installing; then writes `/opt/vedai/bootstrap-ready`. This is a repository-pinned binary version, not a recommendation that it is the latest. Review platform architecture and pinned artifact before using the script on a different host.

`install.sh` requires the bootstrap marker, release and environment file. It extracts to `/opt/vedai/app`, installs `/etc/vedai.env` with root/mode-0600 ownership, removes the staged environment file, creates `/opt/vedai/venv`, installs `app[rag]`, adjusts ownership, installs service units, and starts/enables **the app only**. It deliberately leaves tunnel startup to the operator after origin validation.

## 8.2 Validate origin before publishing

```
sudo systemctl status vedai --no-pager
curl --fail --silent http://127.0.0.1:8765/health
```

If Basic authentication is enabled, use `curl --user USERNAME` and enter the password. Confirm body `status=ready`, expected retrieval mode and generation configuration. Perform one retrieval-only API request and inspect exact source locators. Run available unit tests in a suitable staging/test environment with `.[rag,test]` installed; production `install.sh` installs only `rag`.

### 8.3 Start tunnel and configure host

```
sudo systemctl enable --now vedai-tunnel
sudo journalctl -u vedai-tunnel -n 60 --no-pager
```

Read the generated HTTPS hostname from connector output. Add that hostname to `VEDAI_ALLOWED_HOSTS` in `/etc/vedai.env`, then restart **only the app**:

```
sudo systemctl restart vedai
```

Validate the HTTPS UI, health body and retrieval. Without an allowed hostname the app can reject forwarded requests with HTTP 400. Quick Tunnel URLs are temporary and can change on connector restart. A named tunnel/domain is an infrastructure enhancement requiring separate configuration; it is not configured by these scripts. Preserve the origin hostname when browser-history continuity matters.

## 9. Service behavior and routine operations

`vedai.service` starts the app from `/opt/vedai/app`, supplies `/etc/vedai.env`, uses a five-second on-failure restart, disables bytecode output and Uvicorn access logging, and enables systemd restrictions: no new privileges, private temporary directory, strict system protection and protected home. Its only declared writable runtime path is `/opt/vedai/app/data/index`; the auxiliary passage index can be created there.

`vedai-tunnel.service` forwards loopback over HTTP/2, disables self-update, exposes metrics only at `127.0.0.1:20241`, and restarts after ten seconds. Starting/restarting only `vedai` preserves a running connector and usually preserves its current temporary address. Restarting the connector can produce a new address.

```
sudo systemctl status vedai vedai-tunnel --no-pager
sudo journalctl -u vedai -n 60 --no-pager
sudo journalctl -u vedai-tunnel -n 60 --no-pager
sudo systemctl restart vedai
```

The app does not implement a managed audit log or centralized conversation database. Logs should be reviewed for service errors and accidental secret exposure; do not add request-body logging casually. Provider/infrastructure logs are outside the application access-log setting.

## 10. Code update and rollback

### 10.1 Compatible code-only update

Build the code-only archive, record its SHA-256, transfer it and `deploy/update-code.sh` to `/tmp`, then:

```
sudo bash /tmp/update-code.sh ACTUAL_64_CHARACTER_SHA256
```

The script validates checksum format and archive hash; extracts into `/opt/vedai/releases/<checksum>`; runs `python -m unittest discover -s tests -q` there; saves the current runtime `vedai` package and `pyproject.toml` to `previous-code.tar.gz`; overlays updated files; fixes package ownership; restarts the app; and polls origin health for up to 20 one-second attempts. It leaves corpus/indexes, private environment and the connector intact.

Important limits: the script checks HTTP success on health, not body readiness or a real ask; tests needing absent optional dependencies can be skipped; it does not reinstall changed dependencies; extraction overlays files and does not remove obsolete files; and it does not automatically restore on failure. Follow it with explicit readiness, retrieval and UI checks. Stage dependency or data changes separately.

### 10.2 Manual code rollback

Locate the failed release's `previous-code.tar.gz`, verify the intended rollback target, then during a maintenance window:

```
sudo systemctl stop vedai
sudo tar --extract --gzip --file /opt/vedai/releases/FAILED_CHECKSUM/previous-code.tar.gz --directory
/opt/vedai/app --no-same-owner
sudo chown -R vedai:vedai /opt/vedai/app/vedai
sudo systemctl start vedai
```

Replace `FAILED_CHECKSUM` with the verified release directory. This restores the archived prior code but does not delete newly introduced files or revert dependencies/data. For a change involving deleted/renamed modules or dependency/schema incompatibility, restore a coherent prior release/environment/data snapshot in a staging directory and switch under an operator-reviewed procedure. Verify health **and** retrieval before declaring recovery.

## 11. Validation and release acceptance

| Gate | Required evidence |
|---|---|
| Source | Exact repository commit and inspected change set |
| Artifact | Package manifest, transfer SHA-256 and reviewed file list |
| Data | Coherent manifests, expected counts, raw/lexical integrity verification |
| Tests | Unit/API suite passes with required extras; skipped tests accounted for |
| API | Health body ready, catalog available, retrieval-only ask, source lookup |
| UI | Submit reading, inspect source, save/reopen session, mobile view |
| Generated mode, if enabled | Separately authorized live reading; validate statuses/citations/translation display |
| Operations | Correct Host allowlist, intended authentication, service restart behavior, protected secrets |
| Recovery | Known rollback artifact and backup location; receiving owner assigned |

Optional browser checks from the repository:

```
python -m pip install playwright
python -m playwright install chromium
python tests/browser_smoke.py --url http://127.0.0.1:8765 --output data/runtime/browser-check
python tests/browser_sessions.py --url http://127.0.0.1:8765
```

`browser_smoke.py` explicitly intercepts asks to disable generation. Inspect other live scripts before execution: files such as `tests/live_hosted.py` perform paid end-to-end model calls and require deployment/provider prerequisites. Do not treat unit test completion as a new live model-quality benchmark.

# 12. Troubleshooting

| Symptom | Likely cause | Resolution |
|---|---|---|
| Package import failure | Wrong interpreter or missing extras | Use the intended venv; reinstall `.[rag,test]` for local verification |
| FTS5/trigram/JSON error | SQLite build lacks capability | Replace Python/SQLite build; rerun capability probe |
| `/works` 503 | Corpus not processed / wrong data root | Check working directory, data path and process step |
| Health `indexing_required` | Missing required artifacts | Build all default indexes and inspect manifests |
| Health ready but ask 503 | Stale/malformed artifact or missing DB not fully checked by health | Follow exact error; rebuild a coherent set; run a real ask |
| No generated answer | Key/model absent, provider outage or rejected output | Inspect `generation_status`; read sources; configure provider privately |
| HTTP 400 through tunnel | External hostname absent from allowlist | Add hostname to process environment; restart app |
| HTTP 401 | Basic auth enabled | Supply application username/password; this is not the model API key |
| HTTP 429 | Another query still running | Wait; stop-waiting does not free the server slot |
| Job 404 | Expiry, eviction or restart | Explain lost result; submit a fresh question only when intended |
| Embedding mismatch | Corpus/model/prompt changed | Restore compatible set or rebuild vectors explicitly |
| Source navigation write error | Index directory not writable | Restore intended service ownership/write allowance |
| Session history missing | Different browser profile/origin or cleared storage | Return to the original origin/profile where available; server backup cannot recover it |
| Clipboard unavailable | Browser permission/context restriction | Open source details and select/copy text manually |

# 13. Backup, shutdown and handover

Back up release manifests/commit, pinned source archive/manifest, coherent processed/index artifacts and separately protected environment configuration. Stop reads or use coordinated backup APIs for consistency. Test restore in isolation with database checks, `verify`, selected retriever initialization and retrieval-only API calls. Browser history requires separate user-managed retention; no built-in export is implemented.

For a controlled application stop, use `sudo systemctl stop vedai`. It discards in-memory jobs; warn active readers before planned restarts. To stop the recorded VM when appropriate:

```
gcloud compute instances stop vedai-web --project=tribe-v2-host --zone=us-central1-a
```

VM persistent disk remains allocated; a later connector restart may generate a different temporary hostname. This command is provided for operator use and was not executed during documentation work.

Complete handover by recording actual host/domain, environment owner, key rotation/billing owner, backup location, release/checksum, escalation contact and expert-review responsibility in the receiving team's controlled inventory. No real secrets belong in this document.

# 14. Source references

Authoritative sources are `pyproject.toml`, `.env.example`, `vedai/settings.py`, `vedai/__main__.py`, `vedai/retrieval.py`, `docs/DEPLOYMENT.md`, and all files under `deploy/`. Local verification details are in `Reference/verification.json`; the recorded resolved Python dependencies are in `Reference/environment-versions.txt`.

Pinned deployment scripts